Knapsack Problem

AlgoExpert · Hard · Dynamic Programming

The problem

- Items are [value, weight].
- Returns [[maxValue], [item indices]].

Intuition

- Each item is taken or skipped; a table over items and capacities captures every choice.

Brute force vs optimal

- Try all 2^n subsets
- DP table (items × capacity) → O(n·C)
- 1-D array iterating capacity downward → O(C) space

Approach

- 1. Row per item, column per capacity
- 2. Skip copies the row above
- 3. Take adds the value to the row above at capacity minus weight
- 4. Backtrack to list items

Worked example

	0	1	2	3	4	5	6	7	8	9	10
–	0	0	0	0	0	0	0	0	0	0	0
[1,2]	0	0	1	1	1	1	1	1	1	1	1

Item [1 value, 2 weight]

Worked example

	0	1	2	3	4	5	6	7	8	9	10
–	0	0	0	0	0	0	0	0	0	0	0
[1,2]	0	0	1	1	1	1	1	1	1	1	1
[4,3]	0	0	1	4	4	5	5	5	5	5	5

Item [4 value, 3 weight]

Worked example

	0	1	2	3	4	5	6	7	8	9	10
–	0	0	0	0	0	0	0	0	0	0	0
[1,2]	0	0	1	1	1	1	1	1	1	1	1
[4,3]	0	0	1	4	4	5	5	5	5	5	5
[5,6]	0	0	1	4	4	5	5	5	6	9	9

Item [5 value, 6 weight]

Worked example

	0	1	2	3	4	5	6	7	8	9	10
–	0	0	0	0	0	0	0	0	0	0	0
[1,2]	0	0	1	1	1	1	1	1	1	1	1
[4,3]	0	0	1	4	4	5	5	5	5	5	5
[5,6]	0	0	1	4	4	5	5	5	6	9	9
[6,7]	0	0	1	4	4	5	5	6	6	9	10

Item [6 value, 7 weight]

Worked example

```
max value 10

items [1, 3]
```

Answer: 10 using items [1, 3]

Java solution

```java
public class KnapsackProblem {
    public static List<List<Integer>> knapsackProblem(int[][] items, int capacity) {
        int n = items.length;
        int[][] dp = new int[n + 1][capacity + 1];
        for (int i = 1; i <= n; i++) for (int c = 0; c <= capacity; c++) {
            dp[i][c] = dp[i - 1][c];
            int w = items[i - 1][1];
            if (w <= c) dp[i][c] = Math.max(dp[i][c], dp[i - 1][c - w] + items[i - 1][0]);
        }
        List<Integer> chosen = new ArrayList<>();
        for (int i = n, c = capacity; i > 0; i--)
            if (dp[i][c] != dp[i - 1][c]) { chosen.add(0, i - 1); c -= items[i - 1][1]; }
        return List.of(List.of(dp[n][capacity]), chosen);
    }
}
```

Complexity

- Time: O(n*c)
- Space: O(n*c)

Edge cases & pitfalls

- Pseudo-polynomial in capacity
- 1-D version must iterate capacity downward
- Backtrack to recover items

Interview tips & follow-ups

- Mention unbounded vs 0/1 knapsack

Recap

- Each item is taken or skipped; a table over items and capacities captures every choice.
- Complexity: O(n*c)